

ServiceNow Incident Management

Implementation Document: Client Scripts and UI Policies

This document describes the configuration and implementation of Client Scripts and UI Policies for the Incident table in ServiceNow.

1. Objective

The objective is to improve Incident form behavior by validating user input, dynamically calculating priority, controlling field visibility and mandatory status, and providing additional guidance for critical incidents.

2. UI Policies

2.1 Show Subcategory for Incident

Table: Incident [incident]

Condition: Category is not empty

Reverse if false: True

Field	Mandatory	Visible	Read-only
Subcategory	True	True	False

When a Category is selected, the Subcategory field becomes visible and mandatory. If the Category is cleared, the policy reverses these settings.

2.2 Make Critical Incident Fields Mandatory

Table: Incident [incident]

Condition: Priority is 1 - Critical

Reverse if false: True

Field	Mandatory	Visible	Read-only
Assignment group	True	True	False
Assigned to	True	True	False
Business service	True	True	False

For Priority 1 - Critical incidents, assignment and service information must be completed before the incident can be processed.

3. Client Scripts

3.1 Auto Calculate Incident Priority

Type: onChange

Table: Incident [incident]

Field: Impact

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || isTemplate) return;
    var impact = g_form.getValue('impact');
    var urgency = g_form.getValue('urgency');
    if (!impact || !urgency) return;

    if (impact == '1' && urgency == '1') g_form.setValue('priority', '1');
    else if ((impact == '1' && urgency == '2') || (impact == '2' && urgency == '1'))
        g_form.setValue('priority', '2');
    else if ((impact == '1' && urgency == '3') || (impact == '2' && urgency == '2') || (impact == '3' &&
        urgency == '1')) g_form.setValue('priority', '3');
    else if ((impact == '2' && urgency == '3') || (impact == '3' && urgency == '2'))
        g_form.setValue('priority', '4');
    else if (impact == '3' && urgency == '3') g_form.setValue('priority', '5');
}
```

3.2 Validate Incident Short Description

Type: onSubmit

Table: Incident [incident]

```
function onSubmit() {
    var shortDescription = g_form.getValue('short_description');
    if (!shortDescription || shortDescription.trim() == '') {
        g_form.addErrorMessage('Short description is required before submitting the incident.');
```

3.3 Critical Incident Warning

Type: onChange

Table: Incident [incident]

Field: Priority

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || isTemplate) return;
    if (newValue == '1') {
        g_form.showFieldMsg('priority', 'Critical incident: assignment and service details are required.',
            'warning');
```

3.4 Clear Subcategory When Category Changes

Type: onChange

Table: Incident [incident]

Field: Category

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || isTemplate) return;
    if (oldValue != newValue) g_form.clearValue('subcategory');
    if (!newValue) {
        g_form.setMandatory('subcategory', false);
        g_form.setDisplay('subcategory', false);
    } else {
        g_form.setDisplay('subcategory', true);
        g_form.setMandatory('subcategory', true);
    }
}
```

4. Test Scenarios

Test scenario	Expected result
Category is empty	Subcategory is hidden and not mandatory.
Category is selected	Subcategory is visible and mandatory.
Impact 1 and Urgency 1	Priority becomes 1 - Critical.
Impact 1 and Urgency 2	Priority becomes 2 - High.
Empty short description	Form submission is prevented.
Short description under 10 characters	Validation error is displayed.
Priority is Critical	Assignment and service fields become mandatory.
Category changes	Existing Subcategory value is cleared.

5. Implementation Notes

- Verify the actual choice values for Impact, Urgency, and Priority in the target ServiceNow instance before deployment.
- Use standard Client Scripts for the Incident form. Catalog Client Scripts are not applicable to the standard Incident form.
- Use UI Policies for field visibility and mandatory behavior, and Client Scripts for validation and calculations.
- Test the configuration in a sub-production instance before moving it to production.
- If priority is already controlled by a Data Lookup or other server-side logic, avoid duplicate priority calculations.